

DocuMind RAG Notes

Multi-Document RAG + Document Parsing / Metadata Extraction

1. Big Picture: Humne Exactly Kya Build Kiya?

Humne ek **Metadata-Aware Multi-Document RAG Assistant** build kiya.

Simple words mein:

Ek aisa system jo ek folder ke multiple PDFs ko read karta hai, un PDFs ka metadata samajhta hai, chunks banata hai, embeddings create karta hai, Chroma vector database mein store karta hai, aur user ke question ka answer retrieved context ke basis par deta hai.

Current documents:

```
Module 1-Bootstrap5.pdf
Module 1-CSS3.pdf
Module 1-HTML 5.pdf
Module 1-JavaScript.pdf
Module 2-ANSI SQL Using MySQL.pdf
```

Final output example:

```
Question:
What topics are covered in the CSS3 module?

Answer:
Based on the provided documents, the CSS3 module covers:
- Why CSS?
- CSS Syntax and Comments
- Selectors Playground
- Color & Background Styling
- Typography
- Link and List Styling
- Box Model
- Responsive Web Design with Media Queries

Sources:
Module 1-CSS3.pdf | page=0 | subject=CSS3 | module=1 | title=CSS3 Exercises
```

This means system sirf answer nahi de raha, balki bata bhi raha hai ki answer **kis document, kis page, kis subject, kis module** se aaya.

2. RAG From Scratch Part 1-17 Se Connection

Pehle humne Part 1-17 mein RAG ke individual concepts seekhe:

```
Document loading
Chunking
Embeddings
Vector store
Retriever
Generation
Multi-query
RAG-Fusion
Decomposition
Step-back
HyDE
Routing
Query construction
Advanced indexing
Re-ranking
Agentic RAG
```

Wahan hum mostly concepts ko separately samajh rahe the.

Is project mein humne un concepts ko real-world shape diya.

Pehle hum engine ke parts padh rahe the.
Ab humne un parts se actual vehicle banaya.

3. Concept #5 — Multi-Document RAG System

3.1 Before this concept, problem kya tha?

Basic RAG mein generally ek document ya ek blog se answer aata tha.

Example:

```
Lilian Weng blog → chunks → embeddings → retrieve → answer
```

But real life mein data ek file mein nahi hota. Data folder mein hota hai:

```
HTML notes
CSS notes
JavaScript notes
```

Bootstrap notes
SQL notes
Resume PDFs
Research papers
Company policies
Project reports

Agar user poochta hai:

Compare HTML5, CSS3, Bootstrap5, JavaScript, and SQL.

To answer ek document se nahi banega. System ko multiple documents ke across search karna padega.

Isi problem ko solve karne ke liye **Multi-Document RAG** chahiye.

3.2 Why was Multi-Document RAG needed?

Kyunki real-world RAG system ko sirf ek PDF nahi, **documents ke collection** ko handle karna hota hai.

Agar hum direct LLM ko saare PDFs paste karein:

Problem 1: context window limit
Problem 2: slow response
Problem 3: high cost
Problem 4: no source tracking
Problem 5: har baar full document process karna inefficient

Multi-Document RAG solves this by:

1. PDFs ko once process karo
2. Chunks banao
3. Embeddings create karo
4. Vector DB mein store karo
5. User question ke time sirf relevant chunks retrieve karo
6. LLM ko relevant context do

3.3 Multi-Document RAG actually means kya?

Simple definition:

Multi-Document RAG ek system hai jo multiple documents se relevant chunks retrieve karke LLM ko context deta hai, taaki answer document-grounded ho.

Flow:

```
PDF Folder
↓
Load all PDFs
↓
Split into chunks
↓
Create embeddings
↓
Store in Chroma
↓
User question
↓
Retrieve relevant chunks from all PDFs
↓
LLM answer using retrieved context
↓
Show answer + sources
```

3.4 Real-life story

Imagine tu ek library mein gaya.

Shelf par 5 books hain:

```
HTML5
CSS3
Bootstrap5
JavaScript
SQL
```

Tu librarian se poochta hai:

```
CSS3 module mein kaunse topics hain?
```

Librarian randomly answer nahi deta.

Woh:

1. CSS3 wali book dhoondhta hai
2. Uske relevant pages kholta hai

3. Topics list karta hai
4. Bataata hai page kahan se mila

Yahi Multi-Document RAG karta hai.

4. Concept #1 — Document Parsing / Metadata Extraction

4.1 Before this concept, problem kya tha?

Multi-document RAG sirf chunks retrieve kar sakta hai.

But chunks ke saath agar identity nahi hogi, to source weak dikhega.

Pehle output aisa aa raha tha:

```
Module 1-CSS3.pdf | page=0 | year= | title=
```

Problem:

```
year blank
title blank
subject missing
module number missing
```

System content retrieve kar raha tha, but document ko structured way mein samajh nahi raha tha.

4.2 Document Parsing ki zarurat kyu padi?

Hume har document ka structured metadata chahiye tha.

Course PDFs ke liye useful metadata:

```
ModuleTitle
Subject
ModuleNumber
TopicsCovered
LearningObjectives
KeyConcepts
Summary
```

Example:

```
{
  "ModuleTitle": "CSS3 Exercises",
  "Subject": "CSS3",
  "ModuleNumber": "1",
  "TopicsCovered": [
    "Why CSS?",
    "CSS Syntax and Comments",
    "Selectors Playground",
    "Color & Background Styling"
  ]
}
```

Metadata se source output better ho gaya:

```
Module 1-CSS3.pdf | page=0 | subject=CSS3 | module=1 | title=CSS3 Exercises
```

This is much more professional and trustworthy.

4.3 Original #1 concept kya tha?

Jo #1 document analysis/document parsing concept tha, uska original idea tha:

```
PDF → raw text extraction → LLM prompt → structured JSON
```

Original research-paper type metadata:

```
Paper Title
Publication Year
Authors
Author Contact
Abstract
Summary Abstract
```

This is useful for research papers.

But hamare documents course modules the, research papers nahi.

So humne schema ko change kiya.

4.4 Hamare project mein #1 ka adapted version

Original concept:

Research Paper PDF → LLM → Paper metadata JSON

Hamare project mein:

Course Module PDF → Local parser → Course metadata JSON

Important change:

Humne Groq/LLM parser use nahi kiya final course-module version mein, kyunki course PDFs ke liye filename + first pages se metadata reliably extract ho raha tha, aur Groq quota bhi save karna tha.

So final parser:

Local
Rule-based
No Groq quota
Course-module specific

5. Why #1 and #5 Together Are Powerful

Agar sirf #5 Multi-Document RAG hota:

System answer de sakta tha
But sources metadata-rich nahi hote

Agar sirf #1 Parser hota:

System metadata JSON bana sakta tha
But semantic Q&A nahi kar sakta tha

Combined system:

Parser gives structured identity
RAG gives semantic answering

Together:

PDFs

- └ Parser → subject/module/title/topics
- └ RAG → chunks/embeddings/retrieval/answer

Final result:

Metadata-aware RAG

This is stronger than basic PDF chatbot.

6. What Changed Compared to RAG Part 1-17?

6.1 Same concepts reused

From RAG 1-17, same core concepts used:

- Document loading
- Chunking
- Embeddings
- Vector store
- Retriever
- Prompt
- LLM generation
- Source grounding

Code-level mapping:

- PyPDFLoader = Document loading
- RecursiveCharacterTextSplitter = Chunking
- HuggingFaceEmbeddings = Embeddings
- Chroma = Vector store
- similarity_search = Retrieval
- ChatPromptTemplate = Prompt
- ChatOllama = LLM generation

6.2 New concepts added

This project added practical real-world concepts:

- Folder-based multi-PDF ingestion
- Course-module metadata parser


```
Metadata attached to chunks
Local Ollama LLM
Quota-free answering
CLI project structure
Subject/module/title source display
Manual metadata filtering
GitHub-ready structure
```

6.3 Why these were not enough in basic RAG

Basic RAG can answer:

```
What is task decomposition?
```

But real system needs:

```
Which PDF is this from?
Which module?
Which subject?
Which page?
Can I filter only CSS?
Can I run without API quota?
Can I add more PDFs later?
```

That is why this project extends basic RAG.

7. Code Breakdown — High-Level Structure

Main file:

```
combined_document_intelligence.py
```

Major sections:

```
1. Imports + warnings control
2. Environment config
3. Display helpers
4. Course metadata parser
5. RAG ingestion
6. Vector store creation/loading
7. Retrieval + metadata filtering
```

- 8. Prompt + Ollama answer
- 9. CLI commands

8. Code Section 1 — Imports and Warning Control

Code uses:

```
import argparse
import json
import logging
import os
import re
import shutil
import warnings
from pathlib import Path
from typing import Any, Dict, List, Optional
```

Why?

```
argparse = CLI commands like parse, ingest, ask
json = metadata save/load
os = environment variables
re = regex for module/subject extraction
shutil = delete old chroma_db during rebuild
warnings/logging = clean public output
Path = clean file path handling
typing = better code readability
```

LangChain imports:

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

Meaning:

```
PyPDFLoader = PDF pages load karta hai
RecursiveCharacterTextSplitter = chunks banata hai
```

```
HuggingFaceEmbeddings = text ko vectors mein convert karta hai  
Chroma = vector database  
ChatOllama = local LLM  
ChatPromptTemplate = prompt formatting  
StrOutputParser = final answer string
```

9. Code Section 2 — Environment Config

Important variables:

```
DATA_DIR = Path(os.getenv("DATA_DIR", "./data"))  
CHROMA_DIR = Path(os.getenv("CHROMA_DIR", "./chroma_db"))  
METADATA_DIR = Path(os.getenv("METADATA_DIR", "./data/extracted_metadata"))
```

Meaning:

```
DATA_DIR = PDFs yahan rahenge  
CHROMA_DIR = vector DB yahan save hogi  
METADATA_DIR = parsed JSON metadata yahan save hoga
```

Embedding and LLM config:

```
EMBEDDING_MODEL = os.getenv(  
    "EMBEDDING_MODEL",  
    "sentence-transformers/all-MiniLM-L6-v2"  
)  
  
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3")
```

Meaning:

```
Embedding model local text vectors banata hai  
Ollama model local answer generate karta hai
```

Chunking config:

```
CHUNK_SIZE = 1000  
CHUNK_OVERLAP = 200  
TOP_K = 4  
FETCH_K = 12
```

Meaning:

```
CHUNK_SIZE = har chunk approx 1000 chars
CHUNK_OVERLAP = adjacent chunks mein 200 chars repeat
TOP_K = final context ke liye 4 chunks
FETCH_K = initially 12 candidates retrieve
```

Why $FETCH_K > TOP_K$?

Because system pehle zyada candidates laata hai, phir filtering ke baad top chunks use karta hai.

10. Code Section 3 — Default Course Metadata Schema

```
DEFAULT_COURSE_METADATA = {
    "DocumentType": "course_module",
    "ModuleTitle": "",
    "Subject": "",
    "ModuleNumber": "",
    "TopicsCovered": [],
    "LearningObjectives": [],
    "KeyConcepts": [],
    "Summary": "",
}
```

Why needed?

Agar parser kisi field ko extract nahi kar paata, code break nahi karega. Default blank value use hogi.

This is defensive programming.

11. Code Section 4 — Course Metadata Parser

11.1 `guess_module_number()`

Purpose:

Filename se module number nikaalna.

Example:

```
Module 1-CSS3.pdf → 1
Module 2-ANSI SQL Using MySQL.pdf → 2
```

Logic:

```
match = re.search(r"module\s*(\d+)", filename, flags=re.IGNORECASE)
```

Regex meaning:

```
module word dhoondo  
uske baad number dhoondo
```

11.2 guess_subject_from_filename()

Purpose:

Filename se subject nikaalna.

Example:

```
Module 1-CSS3.pdf → CSS3  
Module 1-Bootstrap5.pdf → Bootstrap5  
Module 2-ANSI SQL Using MySQL.pdf → ANSI SQL Using MySQL
```

Logic:

```
Filename mein "-" ke baad wala part subject hai
```

This works because files follow pattern:

```
Module Number-Subject.pdf
```

11.3 extract_text_from_pdf()

Purpose:

PDF ke first few pages se text extract karna.

Used by parser to detect module title and topics.

Why not full PDF?

Course metadata ke liye generally first pages enough hote hain.

Also faster.

11.4 `extract_numbered_topics()`

Purpose:

PDF text se numbered headings nikaalna.

Example text:

```
1. Why CSS? Inline vs. Internal vs. External
2. CSS Syntax and Comments
3. Selectors Playground
4. Color & Background Styling
```

Output:

```
[
  "Why CSS? Inline vs. Internal vs. External",
  "CSS Syntax and Comments",
  "Selectors Playground",
  "Color & Background Styling"
]
```

Why important?

Ye `TopicsCovered` field banata hai.

11.5 `subject_key_concepts()`

Purpose:

Subject ke basis par known key concepts add karna.

Example:

For CSS3:

```
[
  "CSS syntax",
  "Selectors",
  "Colors and backgrounds",
  "Typography",
  "Box model",
]
```

```
"Tables",  
"Responsive design",  
"Media queries"  
]
```

For Bootstrap5:

```
[  
  "Bootstrap setup",  
  "Containers and rows",  
  "Responsive grid system",  
  "Column classes",  
  "Typography utilities",  
  "Buttons and forms",  
  "Navbars and cards"  
]
```

Why useful?

Agar PDF text extraction incomplete ho, subject-based concepts fallback provide karte hain.

11.6 build_course_metadata()

This is parser brain.

Flow:

```
PDF path  
↓  
Extract text from first pages  
↓  
Guess subject from filename  
↓  
Guess module number from filename  
↓  
Find first meaningful line as module title  
↓  
Extract numbered topics  
↓  
Add key concepts  
↓  
Create learning objectives  
↓  
Create summary
```

↓
Return metadata JSON

Example:

Module 1-CSS3.pdf

Becomes:

```
{
  "DocumentType": "course_module",
  "ModuleTitle": "CSS3 Exercises",
  "Subject": "CSS3",
  "ModuleNumber": "1",
  "TopicsCovered": [...],
  "Summary": "This module focuses on CSS3..."
}
```

12. Code Section 5 — Metadata Save and Load

12.1 `metadata_path_for()`

Creates metadata path:

```
Module 1-CSS3.pdf
↓
data/extracted_metadata/Module 1-CSS3.json
```

12.2 `save_metadata_json()`

Saves metadata to JSON.

Why?

Agar metadata once generated ho gaya, baar-baar parse karne ki zarurat nahi.

12.3 `load_metadata_json()`

Loads existing metadata.

Why?

Future runs fast ho jaate hain.

12.4 `parse_course_pdf_if_needed()`

Logic:

```
If JSON exists and force_parse is false:  
    load existing JSON  
Else:  
    parse again and save JSON
```

This is efficient and practical.

13. Code Section 6 — Metadata Attached to Chroma Chunks

13.1 `flatten_metadata_for_chroma()`

Chroma metadata simple values support karta hai:

```
string  
number  
boolean
```

Lists/dicts directly safe nahi hote.

So:

```
TopicsCovered: ["Selectors", "Typography"]
```

becomes:

```
topics_covered = "Selectors | Typography"
```

Final metadata attached to every chunk:

```
{  
    "file_name": "Module 1-CSS3.pdf",
```

```
"document_type": "course_module",
"module_title": "CSS3 Exercises",
"subject": "CSS3",
"module_number": "1",
"topics_covered": "...",
"key_concepts": "...",
"summary": "..."
}
```

This is the main upgrade over basic RAG.

13.2 `load_pdf_pages_with_metadata()`

Flow:

```
Load metadata
↓
Flatten metadata
↓
Load PDF pages using PyPDFLoader
↓
Attach metadata to every page
```

Important:

```
page.metadata.update(flat_meta)
```

This line makes the project metadata-aware.

Without this line, source output would not have subject/module/title.

14. Code Section 7 — RAG Ingestion

14.1 `load_all_documents_for_rag()`

Loads all PDFs from data folder.

For each PDF:

```
PDF load
Metadata attach
Pages collect
```

Output:

```
Total pages loaded
```

14.2 `split_documents()`

Uses:

```
RecursiveCharacterTextSplitter(  
    chunk_size=1000,  
    chunk_overlap=200  
)
```

Why overlap?

If a concept is split between two chunks, overlap preserves continuity.

Example:

```
Chunk 1 ends with "Bootstrap grid uses..."  
Chunk 2 starts with "rows and columns..."
```

Without overlap, meaning breaks.

With overlap, both chunks preserve context.

14.3 `create_vector_store()`

Creates Chroma DB:

```
Chroma.from_documents(  
    documents=chunks,  
    embedding=get_embedding_function(),  
    persist_directory="chroma_db",  
    collection_name="combined_docs"  
)
```

Meaning:

```
chunks → embeddings → Chroma storage
```

Persist directory means DB saved on disk.

14.4 `load_vector_store()`

If DB already exists, no need to rebuild.

Loads existing Chroma DB.

This makes repeated `ask` commands faster.

14.5 `ingest_documents()`

Command:

```
python combined_document_intelligence.py ingest --rebuild --parse
```

Flow:

```
If --rebuild:
    delete old chroma_db

Load PDFs
Parse/load metadata
Attach metadata
Split into chunks
Create embeddings
Store in Chroma
```

This is indexing phase.

15. Code Section 8 — Retrieval and Metadata Filtering

15.1 `metadata_matches()`

Checks optional filters:

```
subject
module
title
```

Example:

```
--subject CSS3
```

Then only docs where:

```
doc.metadata["subject"] contains CSS3
```

will pass.

15.2 `retrieve_docs()`

Flow:

1. `similarity_search(query, k=FETCH_K)`
2. apply metadata filters
3. if no filtered docs found, fallback to semantic search only
4. return TOP_K docs

Why fallback?

Agar user wrong filter de de:

```
--subject CSS
```

but metadata has:

```
CSS3
```

then strict filter may fail.

Fallback prevents empty answers.

16. Code Section 9 — Prompt and Answer Generation

16.1 `format_docs_for_prompt()`

Creates context like:

```
[Source 1: Module 1-CSS3.pdf, page=0, subject=CSS3, module=1, title=CSS3 Exercises]
PDF chunk content...
```

This gives LLM both content and metadata.

16.2 `answer_question()`

Flow:

```
query
↓
retrieve docs
↓
format docs as context
↓
prompt
↓
Ollama LLM
↓
string answer
```

Prompt instruction:

```
Answer ONLY using the context below.
If the answer is not present, say:
"I don't know based on the provided documents."

If context contains enough information, answer directly.
Do NOT add "I don't know" after giving a valid answer.
```

This fixed the earlier issue where answer was correct but model still added:

```
I don't know based on the provided documents.
```

17. Code Section 10 — Source Printing

`print_sources()` shows:

```
1. Module 1-CSS3.pdf | page=0 | subject=CSS3 | module=1 | title=CSS3
Exercises
  Preview: CSS3 Exercises Theme...
```

This is important for trust.

A RAG system without sources is weak.

A RAG system with source references is much stronger.

18. Code Section 11 — CLI Commands

The project supports commands:

```
python combined_document_intelligence.py parse
```

Parses course metadata.

```
python combined_document_intelligence.py ingest --rebuild --parse
```

Builds vector DB with metadata.

```
python combined_document_intelligence.py ask "What topics are covered in
CSS3?"
```

Asks one question.

```
python combined_document_intelligence.py ask "Explain Bootstrap grid system"
--subject Bootstrap5
```

Asks with metadata filter.

```
python combined_document_intelligence.py chat
```

Interactive chat mode.

19. Errors Faced and Fixes

Error 1 — Ollama not recognized

Error:

```
ollama is not recognized
```

Reason:

Ollama app installed nahi tha ya PATH mein nahi tha.

Fix:

```
Install Ollama  
Restart terminal  
Run ollama --version  
Run ollama pull llama3
```

Error 2 — File not found

Error:

```
can't open file combined_document_intelligence.py
```

Reason:

Project double nested folder mein extract hua tha.

Fix:

```
cd combined_document_intelligence_project
```

inner folder mein jaake run kiya.

Error 3 — Pip installation cancelled

Error:


```
ERROR: Operation cancelled by user
```

Reason:

Installation beech mein cancel ho gayi.

Fix:

```
python -m pip install --upgrade pip setuptools wheel
python -m pip install --no-user --no-cache-dir -r requirements.txt
```

Error 4 — No answer / sources blank

Output:

```
I don't know based on the provided documents.
Retrieved sources:
```

Reason:

`data/` folder mein PDFs nahi the.

Fix:

PDFs directly `data/` folder mein daale.

Then:

```
python combined_document_intelligence.py ingest --rebuild
```

Error 5 — Metadata blank

Old output:

```
year= | title=
```

Reason:

Research paper parser schema course modules ke liye wrong tha.

Fix:

Course schema introduced:

```
ModuleTitle
Subject
ModuleNumber
TopicsCovered
LearningObjectives
KeyConcepts
Summary
```

New output:

```
subject=CSS3 | module=1 | title=CSS3 Exercises
```

Error 6 — Unnecessary “I don't know”

Old output:

```
CSS3 module covers...
I don't know based on the provided documents.
```

Reason:

Prompt unclear tha.

Fix:

Prompt improved:

```
Do NOT add "I don't know" after giving a valid answer.
```

20. Final Working Result

Question:

```
python combined_document_intelligence.py ask "What topics are covered in the
CSS3 module?"
```

Answer:

```
The CSS3 module covers:
- Why CSS?
- CSS Syntax and Comments
- Selectors Playground
- Color & Background Styling
- Typography
- Link and List Styling
- Box Model
- Multiple Columns
- Responsive Web Design with Media Queries
```

Sources:

```
Module 1-CSS3.pdf | page=0 | subject=CSS3 | module=1 | title=CSS3 Exercises
Module 1-CSS3.pdf | page=1 | subject=CSS3 | module=1 | title=CSS3 Exercises
Module 1-CSS3.pdf | page=2 | subject=CSS3 | module=1 | title=CSS3 Exercises
```

This proves:

```
PDFs loaded
Metadata created
Chunks stored
Retriever working
Ollama answering
Sources displayed
```

21. What We Already Knew From RAG 1-17

We already knew:

```
Why chunking is needed
What embeddings are
How Chroma stores vectors
How retriever finds similar chunks
How prompt uses context
Why grounded answers matter
Why metadata filters matter
Why query construction will be useful later
```

So current project did not start from zero.

It reused our RAG foundation.

22. What Was New in This Project

New learnings:

```
How to handle multiple local PDFs
How to design metadata schema based on document type
How to attach metadata to every chunk
How to build local quota-free RAG using Ollama
How to create CLI commands
How to debug real project errors
How to make source output professional
How to prepare project for GitHub
```

23. Why This Project Is Strong

This is not just a PDF chatbot.

It is:

```
Multi-document
Metadata-aware
Local LLM-based
Source-grounded
Extensible
GitHub-ready
```

Current version is a strong base for adding:

```
Query Construction
Routing
Multi-Query
RAG-Fusion
Re-ranking
Decomposition
Step-back
HyDE
Evaluation
Streamlit UI
```

24. Future Connection With RAG Part 1-17

Query Construction

Current:

```
--subject CSS3
```

Future:

```
User: Module 1 ke CSS topics batao
```

System infer karega:

```
{
  "subject": "CSS3",
  "module": "1",
  "semantic_query": "topics covered"
}
```

Routing

Question:

```
What are selectors?
```

Router:

```
CSS3 route
```

Multi-Query

Question:

```
grid system
```

Variants:

Bootstrap grid system
responsive columns
container row col layout
mobile tablet desktop layout

RAG-Fusion

Multiple query results ko rank combine karega.

Re-ranking

Retrieved 12 chunks mein se best 4 select karega.

Decomposition

Question:

Compare HTML5, CSS3, Bootstrap, JavaScript, and SQL.

Break into:

What does HTML5 cover?
What does CSS3 cover?
What does Bootstrap cover?
What does JavaScript cover?
What does SQL cover?

Step-Back

Specific question ke saath broader concept retrieve hoga.

Example:

Why use Bootstrap grid classes?

Step-back:

What is responsive web design?

HyDE

Short query:

grid?

System hypothetical passage generate karega and usse retrieve karega.

25. Memory Hook

Basic RAG finds relevant pages.

Multi-Document RAG finds relevant pages across many books.

Metadata Parsing labels every book and page properly.

Metadata-aware RAG answers with both meaning and identity.

26. Final Summary in My Words

Humne pehle RAG ke parts seekhe the: loading, chunking, embeddings, retrieval, generation, query translation, routing, query construction, reranking, and agentic RAG.

Ab humne un concepts ko real project mein apply kiya.

Project multiple course PDFs ko read karta hai, local parser se subject/module/title/topics metadata banata hai, chunks ke saath metadata attach karta hai, Chroma vector DB mein store karta hai, aur Ollama local LLM se answer generate karta hai.

Earlier output weak tha because metadata blank tha.

Now output strong hai because sources show:

subject=CSS3 | module=1 | title=CSS3 Exercises

This project is now a strong base for advanced RAG improvements.